

UNIDAD 1: Introducción a los lenguajes de marcas

LENGUAJE DE MARCAS Y SISTEMAS DE GESTIÓN DE LA
INFORMACIÓN

Lenguaje de marcas

- ▶ Un lenguaje de marcas es una forma de codificar un documento que, junto con el texto, incorpora etiquetas o marcas que contienen información adicional acerca de la estructura del texto y/o su presentación
- ▶ No son lenguajes de programación
- ▶ Ejemplos:
 - ▶ HTML (HyperText Markup Language)
 - ▶ XML (eXtensible Markup Language)



Un poco de historia

- ▶ En los años 60, IBM intentó resolver sus problemas asociados al tratamiento de documentos en diferentes plataformas:
 - ▶ GML(Generalized Markup Language o Lenguaje de marcas generalizado):
 - ▶ GML libera al creador del documento de preocupaciones específicas del formato del documento tales como especificación de la fuente, línea espaciado, y disposición de página requerida por Script.
 - ▶ Usando GML, un documento está marcado con las etiquetas que definen cuáles son el texto, tablas, etc. El documento se puede entonces ajustar al formato automáticamente para varios dispositivos simplemente especificando un perfil para el dispositivo.
 - ▶ Por ejemplo, es posible ajustar a formato un documento para una impresora laser o para una pantalla simplemente especificando un perfil para el dispositivo sin cambiar el documento



Importancia GML

- ▶ Antes de usar GML, cada aplicación utilizaba sus propias marcas para describir los diferentes elementos.
- ▶ Las marcas son códigos que indican a un programa cómo debe tratar su contenido.
- ▶ Conociendo este sistema y conociendo las marcas de cada aplicación sería posible pasar información de un sistema a otro sin necesidad de perder el formato indicado...
- ▶ **IDEA:** Estandarizar el lenguaje de marcas!



SGML

- ▶ GML y se convirtió en SGML (ISO 8879), en 1986.
- ▶ Esta norma es la que se aplica desde entonces a todos los lenguajes de marcas (HTML, RTF, etc)
- ▶ **SGML** es un metalenguaje que permite definir lenguajes de marcado.
 - ▶ Sintaxis para la inclusión de marcas en los textos y sintaxis del documento que especifica qué etiquetas están permitidas y dónde: el Document Type Definition (DTD).
 - ▶ La definición de la estructura y el contenido de un tipo de documento se realiza por medio de su DTD.
- ▶ Inconvenientes: dificultad en su aprendizaje y uso.



Ejemplo texto SGML

- ▶ <antologia>
 - ▶ <poema><titulo>Soneto Número I</titulo>
 - ▶ <estrofa>
 - <verso>Un soneto me manda hacer Violante,</verso>
 - <verso>que en mi vida me he visto en tanto aprieto;</verso>
 - <verso>catorce versos dicen que es soneto,</verso>
 - <verso>burla burlando van los tres delante.</verso>
 - ▶ </estrofa>
 - <!-- resto del poema -->
 - ▶ </poema>
 - ▶ <!-- otros poemas de la antologia -->
- ▶ <antologia>



Sobre los lenguajes de marcas

▶ Características

- ▶ Texto plano: son editables por cualquier editor ASCII
- ▶ Compactación: ocupa menos
- ▶ Flexibilidad: fácilmente combinable con otros lenguajes
- ▶ Multiplataforma

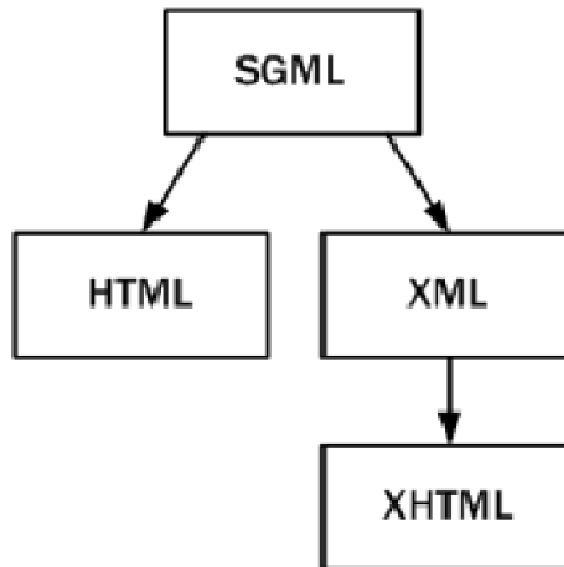
▶ Utilizaciones básicas de los lenguajes de marcas:

- ▶ describir **contenido**, (ej: aplicaciones de bases de datos)
- ▶ definir **formato**, (ej: aplicaciones para formato de texto)
- ▶ **las dos** funciones anteriores indistintamente (ej: HTML)



Evolución Lenguaje de Marcas

- ▶ HTML se crea a partir del SGML.
- ▶ XML surge como respuesta al desorden que supuso el rápido crecimiento del HTML, por lo que podemos decir que su precedencia es SGML



XHTML

- ▶ XHTML es una redefinición del lenguaje HTML utilizando la sintaxis XML.
 - ▶ Por ejemplo, XML:
 - ▶ Obliga a cerrar los tags
 - ▶ Distingue mayúsculas de minúsculas
- ▶ **OBJETIVO:** compatibilidad
 - ▶ Uso de HTML en distintas plataformas, dispositivos, etc.
 - ▶ Vemos [un ejemplo](#)



¿XML sustituye a HTML?

- ▶ XML: lenguaje para creación de aplicaciones y transmisión de datos
- ▶ HTML: lenguaje para creación de páginas web
- ▶ Diferencias:
 - ▶ HTML conjunto de marcas estático frente a XML que tiene marcas abiertas y ampliables
 - ▶ HTML tiene hipervínculos simples (unidireccional y escrito en uno de los documentos implicados) mientras que XML tiene una gestión de hipervínculos avanzada (más de un documento, bidireccional, puede describirse en un documento externo no implicado)
 - ▶ En XML se separa el contenido de la presentación de forma total.



¿Cómo se emplea XML en los programas?

- ▶ Con XML existe un modo estandarizado para que obtengamos la información que necesitamos, no importa como esté estructurada.
- ▶ Si seguimos las reglas especificadas por XML, resultará muy fácil acceder a nuestra información. Esto es así porque existen programas denominados analizadores (*parsers*) que son capaces de leer la sintaxis XML y obtener la información que necesitamos.
- ▶ Podemos utilizar estos analizadores dentro de nuestros propios programas.
- ▶ **¿Pensáis que esto es útil?**



Especificaciones importantes

- ▶ Existen varias especificaciones interrelacionadas que funcionan en conjunto para formar la familia de tecnología XML, y cada especificación cubre un aspecto diferente de la comunicación de la información. Algunas de las más importantes son:
 - ▶ **XML 1.0 y 1.1**: son las especificaciones base a partir de las que se construye la familia: XML. Describe la sintaxis que deben seguir los documentos XML, las reglas que los analizadores XML tienen que cumplir y todo lo que haga falta para leer o escribir un documento
 - ▶ **DTD** y los esquemas nos brindan diferentes métodos para crear plantillas para nuestros tipos de documentos. Podemos lograr que otros documentos, producidos por otros desarrolladores, utilicen estas plantillas y obtener así documentos compatibles con nuestra definición.



-
- ▶ Espacios de nombres, **Namespaces**, es un medio para distinguir entre un vocabulario XML y otro, lo que nos permite crear documentos más consistentes mediante el uso de múltiples vocabularios dentro de un mismo documento.
 - ▶ **XPath** describe un lenguaje de consulta para el direccionamiento de partes de un documento: XML. Esto permite que las aplicaciones puedan requerir una parte específica de un documento XML, Por ejemplo podemos utilizar XPath para obtener todos los apellidos que aparezcan en un documento.
 - ▶ Podemos usar **CSS** (Cascading Style Sheets) para definir la presentación de nuestros documentos. En los casos más complejos podemos utilizar XSL (Extensible Style Sheet Language) que está compuesto por **XSLT** que puede transformar nuestros documentos de un tipo a otro, y Formatting Objects (Objetos de formateo), que se responsabilizan de la información.
-



Etiquetas XML

- ▶ Los documentos XML quedan delimitados por etiquetas:
- ▶ Las etiquetas están entre caracteres `<` y `>` o bien `</` y `>` si son de cierre y tienen un contenido ...
 - ▶ Ej: `<mensaje> Hola </mensaje>`
- ▶ O por una etiqueta de elemento vacío:
 - ▶ `<imagen fichero= "foto.jpg"/>` o bien
 - ▶ `<imagen fichero= 'foto.jpg'/>`
- ▶ Los valores de los atributos de todas las etiquetas deben ir siempre entrecomillados (es válida comilla doble `"` o simple `'`)



Etiquetas XML

- ▶ Las etiquetas distinguen mayúsculas de minúsculas:
 - ▶ `<fecha>` y `<FECHA>` son dos etiquetas distintas
- ▶ Espacios y retornos de carro se tienen en cuenta
- ▶ Los caracteres especiales reservados son: `<`, `>`, `&`, `;`, `"`, `'`
- ▶ Se usan `<`, `>`, `&`, `"`, `'` para poder emplear los caracteres reservados anteriores.
 - ▶ Ej:
 - ▶ `<cliente>`
 - `<empresa> Pepito & Friends </empresa>`
 - ▶ `</cliente>`



Caracteres “no especiales”

- ▶ Para indicar que no se interprete como especiales ninguno de los caracteres de un texto se incluyen: las secciones CDATA.
- ▶ la sección CDATA, se inicia con la cadena `<![CDATA[` y termina con `]]`, hace que no se interprete como etiquetas el texto que contiene.
- ▶ Ej:
 - ▶ `<lista>`
 - ▶ `<nombre>John Doe</nombre>`
 - ▶ `<correo><![CDATA[<jdoe@server.com>]]</correo>`
 - ▶ `</lista>`
- ▶ De forma individual (carácter a carácter), siempre puedo usar una referencia a carácter:
http://en.wikipedia.org/wiki/List_of_XML_and_HTML_character_entity_references



Nombres de elementos o atributos

- ▶ El nombre de un elemento:
 - ▶ Debe empezar por una letra o el carácter subrayado, seguido por letras, dígitos, el carácter punto, el carácter guión o el carácter subrayado
 - ▶ No deberá comenzar por la cadena XML, ya sea en mayúsculas o en minúsculas.
- ▶ ¿Es fácil poner nombres a los elementos y atributos?



Jerarquía XML

- ▶ XML agrupa la información en jerarquías.:
 - ▶ Está formado por elementos
 - ▶ Los elementos de nuestros documentos se ordenan por medio de relación padre/hijo o hermano/hermano. Estos componentes se denominan
- ▶ Por ejemplo `<nombre>`, mostrado jerárquicamente:
 - ▶ `<nombre>`
 - ▶ `<principal>Juan</principal>`
 - ▶ `<apellido1>García</apellido1>`
 - ▶ `<apellido2>Ruiz</apellido2>`
 - ▶ `</nombre>`
- ▶ Esta estructura se denomina árbol.



Jerarquía XML

- ▶ Podemos ver que el elemento nombre tiene a su vez elementos dentro.
- ▶ Ej:
 - ▶ `<nombre>`
 - ▶ `<principal>Juan</principal>`
 - ▶ `<apellido1>García</apellido1>`
 - ▶ `<apellido2>Ruiz</apellido2>`
 - ▶ `</nombre>`
- ▶ El grado de descripción de un elemento que viene dado por las etiquetas de elementos que se incluyen dentro de él, es la granularidad.
- ▶ **¿Podrías disminuir la granularidad del ejemplo? ¿Y aumentarla? ¿Qué ventajas e inconvenientes tiene cada una de las dos opciones?**



Elemento raíz

- ▶ Debe existir obligatoriamente para cada documento XML
- ▶ Se caracteriza por:
 - ▶ Incluir a los restantes elementos
 - ▶ No pertenecer a ningún elemento presente en el documento
 - ▶ El nombre del documento coincide con el nombre del elemento raíz.



Documentos bien formados

- ▶ Estos documentos satisfacen un conjunto de reglas que permiten que una herramienta XML lo analice de forma correcta:
 - ▶ I. Los elementos deben estar anidados adecuadamente: La marca de fin de elemento A no se puede escribir hasta que no se hayan introducido todas las etiquetas de cierre de los elementos cuya marca de inicio esté situada después de la marca de apertura del elemento A.
 - ▶ II. Los valores de los atributos deben encerrarse entre comillas simples o dobles.
 - ▶ III. Todo elemento debe tener una etiqueta de fin o utilizar la etiqueta de elemento vacío.
 - ▶ IV. El elemento debe tener un único elemento raíz.
 - ▶ V. Todo texto debe estar incluido en un elemento.
- ▶ **¿Qué pasa si no lo cumplimos?**
- ▶ **¿Todo documento bien formado es válido?**



- ▶ Los documentos en XML bien formados contienen un prólogo de uno o más elementos:
 - ▶ declaración XML
 - ▶ declaración del tipo de documento XML
 - ▶ información adicional: instrucciones de procesamiento o comentarios.

```
<!-- Inicio del Prólogo -->
<? xml version="1 . 0" standalone="yes"?> <!-- declaración XML -->
< ! DOCTYPE mensaje [ <!-- declaración DOCTYPE -->
    < ! ELEMENT mensaje (#PCDATA) > <!-- declaración ELEMENT -->
]> <!-- final de declaración DOCTYPE -->
<!-- Final del Prólogo -->

<!-- Inicio de los elementos del documento -->
<mensaje>
    Hola, Mundo!
</mensaje>
<!-- Final de los elementos del documento -->
```

Comentarios e instrucciones de proceso

► Comentarios:

- Los comentarios en XML siguen la misma sintaxis que en HTML:

- `<!-- esto es un comentario -->`

► Instrucciones de proceso:

- Por lo que se refiere a instrucciones de proceso, estas permiten especificar en un documento el nombre de una aplicación, así como los datos que se le pretenden pasar como argumentos a la aplicación.
- El parser proporciona esta información al programa que procesa el documento
 - `<?aplicación datos?>`



Declaración XML

- ▶ Se escribe al comienzo del documento con esta sintaxis:
 - ▶ **Version** de especificación XML
 - ▶ **Encoding** : juego de caracteres empleado (por defecto UTF-8) . Podrían ponerse otros, como UTF-16, US-ASCII, ISO-8859-1 (incluye acentos y ñes del castellano) o indicar una referencia a carácter, para el caso que el número sea decimal o hexadecimal respectivamente:
 - ▶ &#código_decimal;
 - ▶ ódigo_hexadecimal;
 - ▶ **Standalone**: campo que indica si el documento está autocontenido, es decir, si no tiene DTD este campo es “yes” y si lleva no es obligatorio porque después se indica cuál es su DTD.
 - ▶ Ej:
`<?xml version="1.0" encoding='ISO-8859-1' standalone="yes"?>`



Declaración del tipo de documento

- ▶ Indica a la aplicación que procesa el documento qué tipo de documento va a procesar. Es **opcional**, la sintaxis es:
 - ▶ `<! DOCTYPE nombre ...>`
 - ▶ Contiene el tipo del elemento raíz del documento y declaraciones de anotaciones especificadas de forma interna al documento, especificadas mediante entidades externas, o de ambas formas.
 - ▶ Ej:
 - **INTERNA:** `< ! DOCTYPE mensaje [ < ! ELEMENT mensaje (#PCDATA) > ] >`
 - **EXTERNA:**
 - PRIVADA: `< ! DOCTYPE mensaje SYSTEM "mensaje . dtd">`
 - PÚBLICA: `< ! DOCTYPE mensaje SYSTEM http://www.abc.org/mens.dtd>`
- **NOTA:** Entraremos más en detalle más adelante...



Ejemplo

```
<!                               Inicio del Prólogo                               >
<? xml version="1 . 0" standalone="yes"?> <!-- declaración XML                -->
< ! DOCTYPE mensaje [           <!-- declaración DOCTYPE -->
    < ! ELEMENT mensaje (#PCDATA) >           <!-- declaración ELEMENT -->
]>                                           <!-- final de declaración DOCTYPE -->
    <!                               Final del Prólogo                               >

    <!                               Inicio de los elementos del documento          >
<mensaje>
    Hola, Mundo!
</mensaje>
    <!                               Final de los elementos del documento          >
```



Atributos

- ▶ Un atributo representa una información complementaria asociada a un elemento. Aparece en la etiqueta de inicio o en la de elemento vacío.
- ▶ Cada elemento puede tener una lista de atributos asociada, en la que el orden es intrascendente pero **no** pueden aparecer atributos repetidos.
- ▶ Un atributo consiste en una pareja de nombre y valor, donde el valor debe aparecer encerrado entre comillas simples o dobles., siguiendo la siguiente sintaxis:
 - ▶ `<elemento atributo1="valor1" atributo2="valor2">`
- ▶ Ej:
 - ▶ `<peso unidad="gramos" precision="0,01">5,73</peso>`



¿Cuándo elemento y cuándo atributo?

- ▶ Ej: Un documento que contiene las transparencias de una presentación. Cada transparencia incluye un título, ¿Se debe crear como elemento o como atributo?
 - ▶ `<transparencia><título>Uso de elementos o atributos</título>...</transparencia>` o bien....
 - ▶ `<transparencia titulo="Uso de elementos o atributos">...</transparencia>`
- ▶ Es necesario saber que los mecanismos de procesamiento y presentación de documentos, permiten tener mejor control sobre los elementos. Por tanto, aquella información que tenga un procesamiento o presentación complejos debe ser un elemento. Teniendo esto en cuenta...
 - ▶ Si la información tiene una estructura interna debe ser un elemento
 - ▶ Si contiene una gran cantidad de información, parece más adecuado, un elemento
 - ▶ Podemos hacer el símil que un elemento es un sustantivo y un atributo un adjetivo
 - ▶ En caso de duda, se debe utilizar un elemento.
- ▶ Aplicando estas pautas podemos concluir que el título de la transparencia debe ser un elemento: tiene carácter sustantivo y debe ser presentado al usuario.

